DSA + LEETCODE MASTER REFERENCE

Tips, Tricks & the December Placement Plan

Prepared for: Nishanth P
EIE, Sri Ramakrishna Engineering College (SREC), Coimbatore
Repo: github.com/nishanth-1431/dsa-in-java
Target: December 2026 Placement Season

"Pattern first, brute force never. GitHub every single day."

TABLE OF CONTENTS

PART 1 — THE PLAN

1.1 The Situation

Placement drives: **~December 2026** — roughly 20-22 weeks from July 2026. Current level: zero problems solved, know Java syntax + basic OOP. Target: Tier 1 mass recruiters guaranteed (TCS / Infosys / Wipro / Cognizant), Tier 2 stretch goal (Zoho / Freshworks) if pace holds.

This is tight but real. This plan deliberately trims the full 11-phase roadmap down to what actually matters for a December drive. Phases 8-11 (Bit/Math/Design, deep SQL, Spring Boot, System Design) are OPTIONAL / PARALLEL — not core. Core DSA (repo folders 01-18) is the priority. Don't feel behind for skipping the rest — Tier 1 companies don't test deep System Design.

1.2 Phase Map → Your Repo Folders

Weeks	Repo Folder(s)	Problems	Priority
1-2	01-Arrays, 02-Strings	20	CORE
3-5	03-Two-Pointers, 04-Sliding-Window, 05-Binary-Search	45	CORE
5-6	Prefix Sum (subfolder in 01-Arrays)	12	CORE
6-8	06-HashMap, 07-Stack, 08-Queue	45	CORE
9-11	09-Linked-List, 10-Recursion, 11-Backtracking	45	CORE
12-14	12-Trees, 13-BST, 14-Trie	40	CORE
15-17	15-Heap, 16-Greedy, 17-Dynamic-Programming	55	CORE (DP light)
18-19	18-Graphs	30	IMPORTANT
—	19-Bit-Manipulation, 20-Math	15	OPTIONAL
Wk6+	SQL (30 min/day, parallel)	—	PARALLEL
20-22	Pure revision + mocks	—	CRITICAL

Total core target: ~290-320 problems by December. That's below the roadmap's full 451, and that's fine — it's the right 300, not all 451.

1.3 Week-by-Week Schedule

Weeks 1-2 (Jul 8 – Jul 21) — 01-Arrays, 02-Strings

- 2 problems/day minimum, Easy-heavy.
- Goal: comfortable with array indexing, string manipulation, basic complexity thinking.
- Commit daily, no exceptions — this builds the habit before problems get hard.

Weeks 3-5 (Jul 22 – Aug 11) — 03-Two-Pointers, 04-Sliding-Window, 05-Binary-Search

- 3 problems/day (these three patterns are the backbone of Tier 1 rounds).
- By end of week 5: instantly recognize 'sorted array + pair' → two pointers.

Week 5-6 overlap — Prefix Sum

- Fold into 01-Arrays as a subfolder 01-Arrays/prefix-sum/.
- 12 problems, ~4 days.

Weeks 6-8 (Aug 12 – Sep 1) — 06-HashMap, 07-Stack, 08-Queue

- 3 problems/day.
- Start SQL practice in parallel now (30 min/day, LeetCode Database track).
- Start TCS NQT / Infosys mock aptitude tests — 1 mock/week from here on.

Weeks 9-11 (Sep 2 – Sep 22) — 09-Linked-List, 10-Recursion, 11-Backtracking

- 3 problems/day.
- Things get genuinely harder here — expect to slow down, that's normal.
- Keep aptitude mocks going weekly.

Weeks 12-14 (Sep 23 – Oct 13) — 12-Trees, 13-BST, 14-Trie

- 3 problems/day. Trees show up in 30%+ of interviews — don't rush this section.
- Start resume + LinkedIn polish this week — don't leave it for November.

Weeks 15-17 (Oct 14 – Nov 3) — 15-Heap, 16-Greedy, 17-Dynamic-Programming

- 2-3 problems/day — DP is intentionally slower pace, quality over quantity.
- DP scope: 1D DP + basic 2D DP only (Unique Paths, LCS, Edit Distance, Knapsack). Skip Advanced DP unless ahead.
- Full mock interview #1 this window (45 min, 2 random problems, no hints).

Weeks 18-19 (Nov 4 – Nov 17) — 18-Graphs

- 2-3 problems/day.
- Focus: Number of Islands, BFS/DFS grid patterns, Course Schedule (topo sort basics).
- Skip Union Find and Dijkstra unless ahead — Tier 1 rarely tests these.

Weeks 20-22 (Nov 18 – Dec 8) — REVISION + MOCK SPRINT

- No new topics. Re-solve 5-8 problems/day from earlier weeks, cold, no notes.
- 2 full mock interviews per week.
- Company-specific practice: TCS NQT format, Infosys HackerRank format (2 problems, 90 min).
- Resume final pass, GitHub README polish, LinkedIn activity visible.

1.4 Daily Routine

Time / Slot	Activity
Weekday session	1.5–2 hrs — DSA (2-3 problems using the solving process)
From Week 6	+30 min SQL
End of session	Update notes/ folder + commit to GitHub
Weekend	Revise 5 problems from the week, no hints. Sunday = 1 mock/aptitude test.

1.5 Repo Hygiene

- One .java file per problem inside its topic folder, named clearly: TwoSum.java, ValidAnagram.java
- Each solution file includes a header comment: Problem / Approach / Time Complexity / Space Complexity
- Update notes/ folder weekly with a short pattern-recap per topic

- Update the progress table in your README (■ → ■) as you clear each topic — recruiters check this
- Commit format: LC [number] - [Problem Name] - [approach]

1.6 Milestone Checkpoints (be honest with yourself here)

Checkpoint	Target
End of August	100+ problems. Arrays/Strings/Two Pointers/Sliding Window/Binary Search/HashMap solid.
End of September	180+ problems. Linked List/Recursion/Backtracking/Trees solid.
End of October	250+ problems. DP basics + Heap/Greedy solid.
Mid-November	290-320 problems. Graphs basics done, into revision mode.
December	Interview-ready for Tier 1, strong shot at Tier 2.

If you fall behind a checkpoint by more than 1-2 weeks, cut Advanced DP and Graphs depth further — not your revision weeks. Revision is what makes problems stick under interview pressure.

1.7 What Not To Do

- Don't jump between topics out of boredom — finish the current folder before moving on.
- Don't skip revision weeks to cram more new topics — recall under pressure is what interviews test.
- Don't compare your daily problem count to classmates — compare to last week's you.
- Don't touch Phase 9-11 (deep SQL, Spring Boot, System Design) until core DSA folders 01-18 are solid.

PART 2 — TIPS & TRICKS

2.1 The Solving Process (run this every single problem)

- **Read twice.** Note input size (n) — it tells you expected time complexity: n ≤ 20 → brute force/backtracking; n ≤ 1,000 → O(n²) fine; n ≤ 100,000 → need O(n log n) or O(n); n ≤ 10^9 → need O(log n) or O(1).
- **Solve by hand on paper first.** No code until you can trace it manually.
- **Pattern-match** — use the cheat sheet in 2.2.
- **Brute force first.** Always. Even if slow.
- **Optimize** — ask 'what work am I repeating?'
- **Code it yourself.** No copy-paste, ever.
- **Write notes.md** — pattern, approach, complexity.
- **Commit to GitHub same day.**

Hard rule: 20-minute struggle timer before any hint.

2.2 Pattern Recognition Cheat Sheet

"What does this problem remind me of?"

Clue in the Problem	Likely Pattern
Sorted array + find a pair/triplet	Two Pointers
Subarray / substring + longest/shortest/contains	Sliding Window
Repeated range sum queries	Prefix Sum
Sorted / rotated array + search	Binary Search
'Find pair that sums to X'	HashMap
Matching brackets / 'next greater element'	Stack (Monotonic Stack)
Fixed-size window max/min	Deque (Monotonic Queue)
'All combinations/subsets/permutations'	Backtracking
Tree height/path/sum questions	DFS on Tree
Level-by-level tree processing	BFS on Tree
'Sorted output from a tree'	BST Inorder Traversal
Autocomplete / prefix matching	Trie
Grid flood-fill / 'number of islands'	DFS/BFS on Grid
'Minimum steps/shortest path' unweighted	BFS
'Shortest path' with weights	Dijkstra (min-heap)
'Connected components' / 'same group'	Union Find
'Kth largest/smallest'	Heap (PriorityQueue)
'Maximize/minimize', greedy provable	Greedy
Overlapping subproblems + optimal substructure	Dynamic Programming

Clue in the Problem	Likely Pattern
'In-place, O(1) space' hint on array	Bit Manipulation / swapping

2.3 Java-Specific Syntax Tricks

```
// HashMap counting pattern
map.put(key, map.getOrDefault(key, 0) + 1);

// HashMap computeIfAbsent (for lists as values)
map.computeIfAbsent(key, k -> new ArrayList<>()).add(value);

// Stack -- always use ArrayDeque, NOT java.util.Stack
Deque<Integer> stack = new ArrayDeque<>();
stack.push(x); stack.pop(); stack.peek();

// Min-Heap (default)
PriorityQueue<Integer> minHeap = new PriorityQueue<>();

// Max-Heap
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());

// Max-Heap with custom comparator (e.g., by frequency)
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> b[1] - a[1]);

// Sorting with comparator
Arrays.sort(arr, (a, b) -> a[0] - b[0]); // 2D array sort by column 0

// Common constants
Integer.MAX_VALUE, Integer.MIN_VALUE

// 2D array init
int[][] dp = new int[m][n];
for (int[] row : dp) Arrays.fill(row, -1);

// StringBuilder for string building (never += in a loop)
StringBuilder sb = new StringBuilder();
sb.append(x); sb.reverse(); sb.toString();

// Character frequency array (faster than HashMap for lowercase letters)
int[] freq = new int[26];
freq[c - 'a']++;
```

2.4 Bit Tricks (memorize these cold)

```
n & (n - 1)      // removes the lowest set bit
n & 1            // checks odd (1) or even (0)
n & -n           // isolates the lowest set bit
a ^ a == 0       // XOR cancels identical pairs -- used in Single Number
n & (n-1) == 0   // checks if n is a power of two
```

2.5 Recursion / Backtracking Template

```
void backtrack(State state, List<Result> results) {
    if (isBaseCase(state)) {
        results.add(new Result(state)); // save a copy!
        return;
    }
    for (Choice choice : getChoices(state)) {
        state.choose(choice);      // choose
        backtrack(state, results); // explore
        state.unchoose(choice);    // un-choose (backtrack)
    }
```

```
}
```

2.6 Sliding Window Template (variable size)

```
int left = 0;
for (int right = 0; right < n; right++) {
    // expand: add arr[right] to window state
    while (windowIsInvalid()) {
        // shrink: remove arr[left] from window state
        left++;
    }
    // update answer using current window [left, right]
}
```

2.7 Binary Search Template (classic + 'on answer space')

```
// Classic
int lo = 0, hi = n - 1;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2; // avoids overflow
    if (arr[mid] == target) return mid;
    else if (arr[mid] < target) lo = mid + 1;
    else hi = mid - 1;
}

// Binary Search on Answer Space (e.g., Koko Eating Bananas)
int lo = minPossibleAnswer, hi = maxPossibleAnswer;
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (isFeasible(mid)) hi = mid;   // try smaller
    else lo = mid + 1;               // need bigger
}
// lo is the answer
```

2.8 DP Approach (never skip these steps)

- Write the brute-force **recursive** solution first (even if exponential).
- Identify repeated subproblems — add a **memo** (top-down).
- Convert to **tabulation** (bottom-up) once memo works.
- Optimize space if possible (rolling array, e.g., dp[i-1] and dp[i-2] only).

Ask in words: 'What does dp[i] represent? What earlier states does it depend on?' If you can't answer this in one sentence, you don't understand the problem yet — go back to step 1.

2.9 Common Mistakes That Cost Marks in Interviews

- Not asking clarifying questions before coding (empty array? duplicates? negative numbers?)
- Jumping to code before explaining your approach out loud
- Ignoring edge cases: empty input, single element, all same elements, negative numbers
- Not stating time/space complexity at the end — always say it out loud
- Using java.util.Stack instead of ArrayDeque (works, but shows you don't know Java internals)
- Off-by-one errors in binary search / two pointers — always trace through a tiny example after coding

2.10 Mock Interview Structure (practice this exact flow)

1. Restate the problem in your own words (30 sec)
2. Ask 1-2 clarifying questions

3. Say your approach out loud BEFORE coding ('I'll use two pointers because...')
4. Code it, talking through what you're writing
5. Trace through the example by hand
6. State time & space complexity
7. Mention edge cases you handled

2.11 GitHub Commit Format (non-negotiable)

```
LC [number] - [Problem Name] - [approach]
Example: LC 1 - Two Sum - HashMap

SPRING - [pattern] - [what you built]
SQL - [problem] - [approach]
```

Source roadmap: Java Full Stack + DSA Mastery Roadmap v2 — refer back to it for the full problem list per phase. This plan is a trimmed, December-scoped version for execution.